



UNIVERSITI TUNKU ABDUL RAHMAN DU012(A)

Wholly owned by UTAR Education Foundation

(200201010564(578227-M))

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY (FICT)

JUNE 2026 TRIMESTER

BACHELOR OF COMPUTER SCIENCE (HONOURS)

UCCD2063 ARTIFICIAL INTELLIGENCE TECHNIQUES

Name: DESMOND BEH

Student ID: 2402587

Title: SARSA REINFORCEMENT LEARNING REPORT

1. Reinforcement Learning Problem Calculation

The reinforcement learning problem was formulated using a platformer environment developed in Godot Engine. The objective of the agent is to interact with the environment and learn a suitable sequence of actions to collect all available apples while avoiding dangerous situations such as contact with the snail enemy.

The environment follows a **Markov Decision Process (MDP)** consisting of states, actions, rewards and transitions [1]. At each interaction step, the Godot environment provides the current state to the SARSA agent. The agent then selects an action according to its current policy. The selected action is transmitted to the Godot environment, where the player performs the corresponding movement. The resulting state and reward are then returned to the agent.

The learning process is represented by the interaction:

State → Action → Reward → Next State → Next Action

This process continues until the episode reaches a terminal condition. An episode terminates when the player successfully collects all apples or encounters a failure condition such as being hit by an enemy.

The SARSA approach was selected as the reinforcement learning method because it is an on-policy temporal-difference learning algorithm [2]. Unlike methods that estimate the value of the best possible next action, SARSA updates the current state-action value based on the action that the agent actually selects under its current policy [2].

The main objective of the agent can therefore be represented as maximising the accumulated reward over an episode:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where G_t represents the return from time step, R represents the received reward and γ represents the discount factor.

2. Environment and State Representation

2.1 Godot Environment

The reinforcement learning environment was implemented in Godot Engine. The environment contains the player character, collectible apples, platforms and snail enemies. The Godot environment is responsible for executing the actions selected by the SARSA agent and returning the resulting environmental information.

Communication between the Godot environment and the Python SARSA agent was implemented using a **TCP connection**. The Python program operates as the SARSA server while Godot acts as the environment that sends state information and receives actions.

The communication process consists of three main types of messages:

- i. Reset message – informs the SARSA agent that a new episode has started.
- ii. Step message – provides the resulting state, reward and termination status after an action.
- iii. Action message – sends the selected action from the SARSA agent to Godot.

The state information is represented as a string containing several discrete values.

For example: **0_0_0_0_1_1**, each value corresponds to a particular environmental characteristic.

2.2 State Representation

The environment uses six discrete state variables:

State Variable	Description
apples_collected (A)	Number of apples collected by player
apple_direction_x (D_x)	Horizontal direction of the nearest apple
apple_direction_y (D_y)	Vertical direction of the nearest apple
vertical_state (V)	Current vertical movement of the player

grounded (G)	Indicates whether the agent is on the floor
snail_danger (D _s)	Indicates whether a snail is within the specific danger range

The complete state is represented using:

$$S = (A, D_x, D_y, V, G, D_s)$$

For example, the state:

0_1_0_0_1_0

can be interpreted as a state where no apples have been collected, the nearest apple is located to the right of the player, the apple is at approximately the same vertical level, the player has no significant vertical movement, the player is on the floor and there is no detected snail danger.

The state representation converts continuous information from the game environment into a smaller number of discrete states. This allows the SARSA agent to maintain a Q-table where each state-action pair has an associated value.

3. Action and Reward Design

3.1 Action Space

The SARSA agent uses six possible actions:

Action ID	Action
0	Idle
1	Move Left
2	Move Right
3	Jump
4	Move Left + Jump
5	Move Right + Jump

The inclusion of combined movement actions allows the agent to perform more complex platformer movements. For example, right + jump allows the player to move towards an apple while simultaneously changing its vertical position.

The six-action design is represented as:

$$A = \{0,1,2,3,4,5\}$$

At every decision step, SARSA selects one action from this action space using an **epsilon-greedy policy**.

3.2 Reward Design

A reward system was implemented to encourage the agent to collect apples while discouraging unnecessary movement and dangerous interactions.

The main reward conditions are:

Event	Reward
Normal time step	-0.05
Collect one apple	+50
Collect all apples	+200
Hit by snail	-100

The small negative reward of **-0.05 per step** acts as a time penalty. This encourages the agent to complete the task using fewer steps rather than continuously moving around the environment.

When an apple is collected, the agent receives a positive reward of **+50**. When the final apple is collected, an additional larger reward of **+200** is provided and the episode terminates successfully.

A negative reward is provided when the player encounters a snail. This makes dangerous states less desirable during learning.

The reward values allow the Q-table to gradually associate successful behaviours with higher action values.

4. SARSA Algorithm and Configuration

4.1 SARSA Learning Process

SARSA is an on-policy temporal-difference reinforcement learning algorithm. The name SARSA is derived from the sequence:

State → Action → Reward → State → Action

The algorithm begins by obtaining current state S_t and selecting an action A_t using epsilon-greedy policy. After the action is executed, the environment returns a reward R_{t+1} and a new state S_{t+1} . The agent then selects the next action A_{t+1} .

The Q-value is updated using:

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$ [3], where

- $Q(S_t, A_t)$ = current Q-value
- α = learning rate
- R_{t+1} = reward received
- γ = discount factor
- $Q(S_{t+1}, A_{t+1})$ = Q-value of the next state-action pair

An important characteristic of SARSA is that the **next action is selected before the Q-value update**. [3]

Therefore, the update considers the action that the current policy actually intends to perform.

4.2 Epsilon-Greedy Policy

An epsilon-greedy strategy was used to balance exploration and exploitation.

During exploration, the agent selects an action randomly. During exploitation, it selects the action with the highest Q-value for the current state.

The initial exploration rate was:

$\epsilon = 1.0$

The minimum exploration rate was:

$\epsilon_{\min} = 0.05$

The exploration rate was gradually reduced using:

$\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \times 0.995)$

Therefore, the agent performs substantial exploration during the early training episodes and increasingly relies on learned Q-values as training progresses.

4.3 SARSA Configuration

The main configuration used for training was:

Parameter	Value
State size	6
Action size	6
Learning rate (α)	0.1
Discount factor (γ)	0.9
Initial epsilon	1.0
Minimum epsilon	0.05
Epsilon decay	0.995
Number of training episodes	1000
Q-table file	sarsa_q_table.json
Communication host	127.0.0.1
Communication port	9999
Random seed	42

The learning rate of 0.1 allows the Q-values to gradually incorporate new experience without completely replacing previous knowledge.

The discount factor of 0.9 gives significant importance to future rewards. This is particularly relevant because the agent may need to perform several actions before reaching an apple.

A fixed random seed of 42 was also used to improve reproducibility of the training process.

5. Training and Evaluation Procedure

5.1 Training Procedure

The SARSA agent was trained for **1,000 episodes**. Each episode began by resetting the Godot environment and obtaining the initial state.

The training procedure followed these steps:

1. Start the Python SARSA server.
2. Establish a TCP connection with Godot.
3. Reset the game environment.
4. Receive the initial state.
5. Select an initial action using the epsilon-greedy policy.
6. Send the action to Godot.
7. Receive the next state, reward and termination status.
8. Select the next action.
9. Perform the SARSA Q-value update.
10. Continue until the episode terminates.
11. Record the episode reward and number of steps.
12. Reduce epsilon.
13. Save the Q-table periodically.
14. Reset the environment for the next episode.

The Q-table was stored in a JSON file so that the learned state-action values could be preserved after training.

5.2 Training Data Recording

During training, the following information was recorded for each episode:

- Episode number
- Total episode reward
- Number of steps
- Epsilon value

The recorded data was stored in: **sarsa_training_log.csv**.

This training log was subsequently loaded using Pandas for analysis and visualisation.

The recorded reward values can be used to determine whether the agent's performance changes throughout the training process. Similarly, the number of steps per episode can be examined to determine whether the agent becomes more efficient over time.

6. Result and Performance Analysis

6.1 Episode Reward

[Refer to Appendix – Figure A6]

The total reward obtained by the SARSA agent was recorded for each of the 1,000 training episodes. During the early stages of training, the agent performed a large amount of exploration because the initial epsilon was 1.0. Consequently, the actions selected by the agent were highly variable. This resulted in considerable variation in episode rewards.

As training progressed, epsilon gradually decreased according to the specified decay rate. The agent therefore increasingly relied on the Q-values learned from previous interactions.

The highest recorded episode reward was **237.65**, while the lowest recorded reward was **-323**.

The average reward across the 1,000 episodes was **-203.89**.

The reward graph should be analysed based on its overall trend rather than a single episode. A gradual increase or reduced variability in reward would indicate that the agent is learning more consistent behaviour.

6.2 Number of Steps per Episode

[Refer to Appendix – Figure A7]

The number of steps required to complete each episode was recorded.

The number of steps provides another measurement of agent behaviour. An episode with a very large number of steps indicates that the agent spent a longer time interacting with the environment before reaching a terminal condition. The time penalty of -0.05 per step encourages the agent to avoid unnecessarily long episodes.

Therefore, a reduction in the number of steps together with improved rewards would indicate that the agent is becoming more efficient.

The average number of steps per episode was **70.84**.

6.3 Epsilon During Training

[Refer to Appendix – Figure A8]

The exploration rate decreased from the initial value of 1.0 towards the minimum value of 0.05.

At the beginning of training, the high epsilon value caused the agent to explore different actions extensively. This was useful because the Q-table initially contained little or no learned information.

As the number of episodes increased, epsilon decreased. Consequently, the agent increasingly selected actions based on the highest learned Q-value instead of selecting random actions.

After 1,000 episodes, the epsilon value was approximately **0.05**.

6.4 Q-table Development

[Refer to Appendix – Figure A4]

The SARSA agent stored its learned action values in the Q-table. Each state was associated with six possible action values corresponding to the six available actions.

For example, a state may have a Q-table entry similar to:

State: 0_1_0_0_1_0, in which

Idle → Q-value

Move Left → Q-value

Move Right → Q-value

Jump → Q-value

Move Left and Jump → Q-value

Move Right and Jump → Q-value

The agent uses these values to determine which action is preferable for a particular state.

During training, the number of states represented in the Q-table increased as the agent encountered different environmental conditions. The final Q-table contained **0_0_0_-1_0_1 states** after the 1,000 training episodes.

The Q-table therefore represents the policy learned by the SARSA agent from its interaction with the Godot environment.

Reference - SARSA Reinforcement Learning

- [1] Z. Ding, Y. Huang, H. Yuan, and H. Dong, "Introduction to Reinforcement Learning," *Deep Reinforcement Learning*, pp. 47–123, 2020, doi: 10.1007/978-981-15-4095-0_2.
- [2] G. Yao, N. Zhang, Z. Duan, and C. Tian, "Improved SARSA and DQN algorithms for reinforcement learning," *Theoretical Computer Science*, vol. 1027, p. 115025, Dec. 2024, doi: 10.1016/j.tcs.2024.115025.
- [3] P. Sharma, N. P. Padhy, and R. Singh, "A Comparative Study of SARSA and Q-Learning for Cost-Effective Microgrid Operations," *2025 IEEE 5th International Conference on Sustainable Energy and Future Electric Transportation (SEFET)*, pp. 1–6, July 2025, doi: 10.1109/sefet65155.2025.11255235.

Appendix – SARSA Reinforcement Learning

LLM Prompt URL: <https://chatgpt.com/share/6a91761f-0dd4-83ec-a5f9-850363ecc188>

Figure A1: Godot platformer environment used for SARSA training

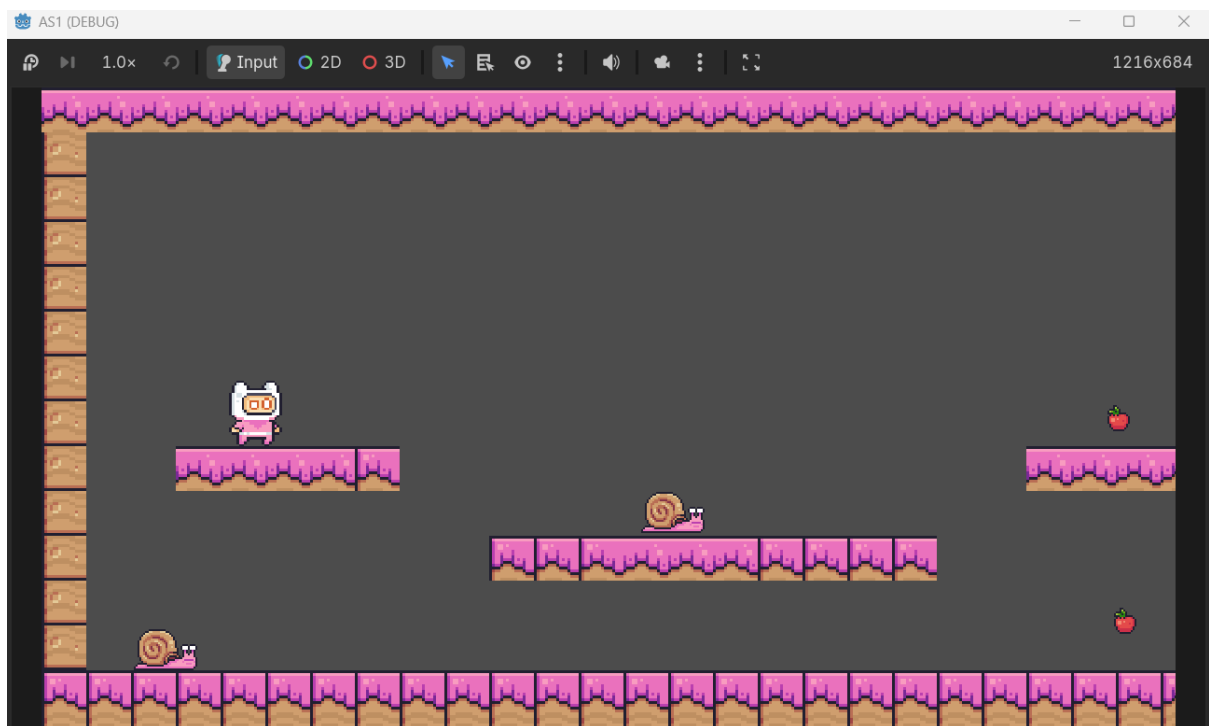


Figure A2: SARSA training sample output showing episode interaction and rewards

```
State: 0_0_0_0_0 | Reward: -0.05 | Done: False
Sent action: 0 idle
State: 0_0_0_1_0_0 | Reward: -0.05 | Done: False
Sent action: 4 left+jump
State: 0_0_0_1_0_0 | Reward: -0.05 | Done: False
Sent action: 4 left+jump
State: 0_0_0_0_1_0 | Reward: -0.05 | Done: False
Sent action: 4 left+jump
State: 0_0_0_1_0_0 | Reward: -0.05 | Done: False
Sent action: 5 right+jump
State: 0_0_0_0_0_0 | Reward: -0.05 | Done: False
```

Figure A3: SARSA training sample output near the end of the 1,000-episode training process

```

=====
Episode 1000/1000
Initial state: 0_0_0_0_1
Initial action: 3 jump
Sent action: 3 jump

Q-table saved successfully.
File: sarsa_q_table.json
Number of states: 16

Training log saved successfully.

```

Figure A4: Sample data from Q-table - sarsa_q_table.json

State	÷ 0	÷ 1	÷ 2	÷ 3	÷ 4	÷ 5
0_0_0_0_1_1	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999
0_0_0_1_0_1	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999
0_0_0_-1_0_1	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999
0_0_0_0_0_1	-0.499998	-0.499999	-0.499998	-0.499998	-0.499999	-0.499999
0_0_0_0_1_0	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999
0_0_0_-1_0_0	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999
0_0_0_1_0_0	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999
0_0_0_0_0_0	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999	-0.499999
1_0_0_0_1_0	-0.023558	-0.024612	-0.035441	-0.031723	-0.026480	-0.025964
1_0_0_1_0_0	-0.023846	-0.026937	-0.023884	-0.026858	-0.033275	-0.029241
1_0_0_0_0_0	-0.011237	-0.014536	-0.009950	-0.010909	-0.010915	-0.015689
1_0_0_-1_0_0	-0.030691	-0.038115	-0.027094	-0.037073	-0.034939	-0.030252
1_0_0_1_0_1	-0.005000	0.000000	-0.009500	0.000000	0.000000	0.000000
1_0_0_-1_0_1	0.000000	0.000000	-0.006337	0.000000	0.000000	0.000000
2_0_0_1_0_0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
0_0_0_1_1_1	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

Figure A5: Sample data from CSV File - sarsa_training_log.csv

÷	episode	÷	reward	÷	steps	÷	epsilon	÷	
	0		1		-200.65		13		0.995000
	1		2		-203.30		66		0.990025
	2		3		-203.20		64		0.985075
	3		4		-206.85		137		0.980150
	4		5		-204.40		88		0.975249
	...		...		...		...		...
	994		995		-203.30		66		0.050000
	995		996		-202.00		40		0.050000
	996		997		-205.40		108		0.050000
	997		998		-201.05		21		0.050000
	998		999		-201.30		26		0.050000

Figure A6: Episode reward over 1000 training episodes

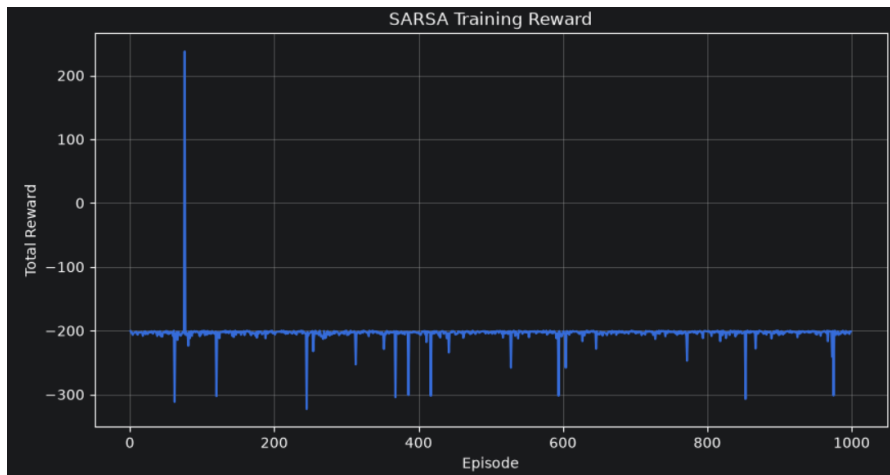


Figure A7: Number of steps per episode

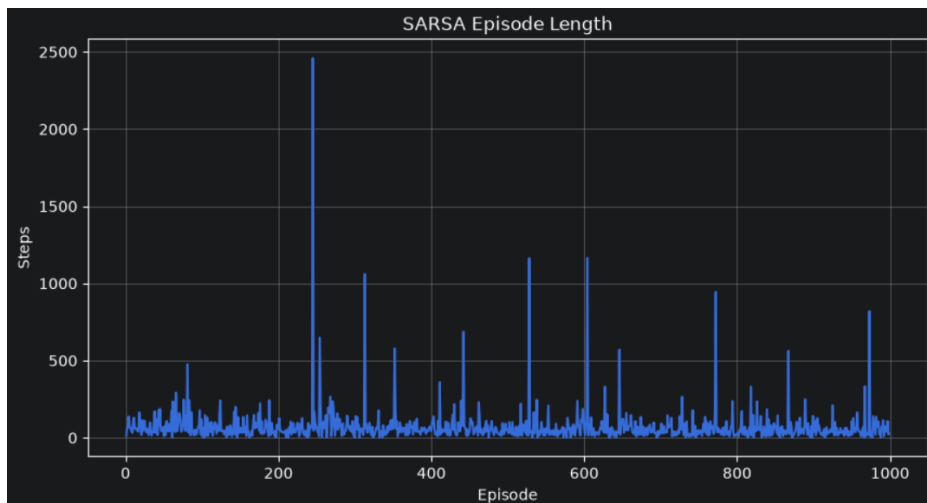


Figure A8: Epsilon decay during training

